Git Auto-Commit Watcher

A production-quality, fully local Git automation tool for Windows that watches your repository for file changes and automatically creates timestamped commits and pushes them to GitHub — with no AI API, cloud service, or subscription required.

Features

- **Event-driven** — Uses `watchdog` (Windows `ReadDirectoryChangesW`) for instant detection without polling.
- **Debounced** — Consolidates burst saves from a single editor operation into one commit (default 1.5 s).
- **Distinct states** — Each separately detected saved state gets its own commit.
- **Idle-safe** — Zero new commits or pushes when nothing changes.
- **Secret-aware** — Blocks `.env`, `*.pem`, `*.key`, and other credential files; scans diff for password/token patterns before committing.
- **Resilient push** — Retries transient network failures with bounded exponential back-off; classifies permanent errors and stops retrying immediately.
- **Restart recovery** — Persists pending-push state so no commit is silently lost across restarts.
- **Non-destructive** — Never force-pushes, resets, or rewrites history.

Prerequisites

Requirement	Version
Python	3.10 +
Git	2.30 +
watchdog	4.0 + (installed automatically)
gitpython	3.1 + (installed automatically)

Installation

```
# From the repository root
pip install -r requirements.txt
```

Verify:

```
python -c "import watchdog, git; print('OK')"
```

Quick Start

```
# Start the watcher (watches the current directory)
python run_watcher.py

# Or point it at another repository
python run_watcher.py --repo "D:\PROJECTS\my_other_repo"

# Verbose / debug output
python run_watcher.py --verbose
```

Stop: press `Ctrl+C` — the watcher shuts down cleanly.

Configuration

Create `autocommit_config.json` in the repository root to override defaults:

```
{
  "debounce_seconds": 1.5,
  "push_retry_base_seconds": 5,
  "push_retry_max_seconds": 300,
  "push_retry_max_attempts": 10,
  "target_branch": "",
  "remote": "origin",
  "commit_message_template": "Auto-save: %Y-%m-%d %H:%M:%S",
  "verbose": false
}
```

All settings can also be set via environment variables prefixed with `AC_`:

```
$env:AC_DEBOUNCE_SECONDS = "2.0"
$env:AC_VERBOSE = "true"
python run_watcher.py
```

Settings Reference

Key	Default	Description
`debounce_seconds`	`1.5`	Wait this many seconds after the last event before committing
`push_retry_base_seconds`	`5`	Initial delay between push retries
`push_retry_max_seconds`	`300`	Maximum delay between push retries

Key	Default	Description
`push_retry_max_attempts`	`10`	Max retry attempts (0 = unlimited)
`target_branch`	`""`	Remote branch to push to (empty = auto-detect)
`remote`	`"origin"`	Git remote name
`commit_message_template`	`"Auto-save: %Y-%m-%d %H:%M:%S"`	Python `strftime` template
`verbose`	`false`	Enable DEBUG logging

How It Works

```
File saved
    │
    ▼ (event from watchdog ReadDirectoryChangesW)
Filter: ignore .git/*, __pycache__/*, *.pyc, .env, etc.
    │
    ▼
Debounce timer resets (1.5 s window)
    │
    ▼ (timer fires)
Commit Worker thread wakes
    │
    ├─ git add -A
    ├─ Unstage blocked filenames (.env, *.pem, *.key, …)
    ├─ Check: anything staged? → No → skip
    ├─ Scan diff for secret patterns → match → ABORT (unstage all)
    ├─ git commit -m "Auto-save: YYYY-MM-DD HH:MM:SS"
    └─ git push (with exponential back-off for transient errors)
```

A single `threading.Lock` serialises all Git operations — concurrent filesystem events cannot create competing commits.

Security

Files that are **never staged**, regardless of `.gitignore`:

- `.env`, `.env.*`
- `*.pem`, `*.key`, `*.p12`, `*.pfx`, `*.crt`, `*.cer`
- `id_rsa`, `id_ed25519`, `id_ecdsa`
- `secrets.json`, `secrets.yaml`, `credentials.json`, `credentials.yaml`
- `auth.json`, `*.gpg`

Content patterns that **abort a commit** (the entire staging is rolled back):

- `PASSWORD=…`, `SECRET=…`, `TOKEN=…`, `API_KEY=…`
- `-----BEGIN RSA PRIVATE KEY-----` and similar PEM headers
- AWS access key format (`AKIA…`)

Branch and Remote Safety

On startup the watcher:

1. Reads the current local branch (`git symbolic-ref HEAD`).
2. Reads the remote's default branch (`remotes/origin/HEAD`).
3. If they differ (e.g., local `master` vs remote `main`), it **logs a warning** and uses the remote default as the push target — no renaming, no deleting, no force-pushing.
4. If the local branch has no upstream yet, it sets `-u origin <branch>` on the first push.

Push Retry and Recovery

Error type	Behaviour
Transient (connection reset, timeout)	Retry with exponential back-off up to `push_retry_max_attempts`
Permanent (auth failure, non-fast-forward, repo not found)	Stop retrying immediately; log error; keep local commits safe

Pending pushes survive restarts: `autocommit_state.json` (gitignored) records whether a push is pending. On the next startup the watcher attempts a recovery push before watching for new changes.

Windows Startup (Run on Login)

Option A — Task Scheduler (recommended)

1. Open **Task Scheduler** → *Create Basic Task*
2. **Trigger:** At log on
3. **Action:** Start a program
   - Program: `C:\Program Files\Python312\python.exe`
   - Arguments: `"D:\PROJECTS\code_dumper\run_watcher.py" --repo "D:\PROJECTS\code_dumper"`
   - Start in: `D:\PROJECTS\code_dumper`
4. Check *Run whether user is logged on or not* if you want background operation.

Option B — Startup folder shortcut

Create a `.bat` file in `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup`:

```
@echo off
python "D:\PROJECTS\code_dumper\run_watcher.py" --repo
```

```
"D:\PROJECTS\code_dumper" >> "D:\PROJECTS\code_dumper\watcher.log" 2>&1
```

Troubleshooting

"fatal: 'origin' does not appear to be a git repository"

Your remote URL is wrong or the remote doesn't exist yet. Run `git remote -v` and correct the URL.

Push permanently failed (auth / non-fast-forward)

The watcher stops pushing (but keeps committing locally). Fix the cause:

- **Auth:** check GitHub credentials via `git push` manually.
- **Non-fast-forward:** someone pushed to the remote branch. Pull changes with `git pull --rebase`, then delete `autocommit_state.json` and restart.

Too many commits for a single save

Increase `debounce_seconds` in `autocommit_config.json` to 2–3 seconds.

Sensitive file blocked accidentally

If a legitimate file is being blocked, check if its name matches any pattern in `autocommit/config.py`:`BLOCKED_FILENAME_PATTERNS` and remove or adjust the pattern.

Watcher exits unexpectedly

Check the console output. The most common cause is a Python exception during startup (missing `.git`, wrong `--repo` path).

Running Tests

Tests use isolated temporary Git repositories — **no commits are ever made to your real repository**.

```
# Run all tests
python -m pytest tests/test_autocommit.py -v

# Run only fast unit tests (skip watcher integration)
python -m pytest tests/test_autocommit.py -v -k "not TestDebounce"
```

Project Structure

```
code_dumper/
├── autocommit/
```

```
│   ├── __init__.py        # Package init
│   ├── config.py          # Settings, secret patterns, ignore rules
│   ├── git_ops.py         # All Git operations (commit, push, scan)
│   ├── state.py           # Persistent push-state manager
│   └── watcher.py         # Filesystem observer + commit worker
├── tests/
│   └── test_autocommit.py # Full test suite (no network required)
├── .gitignore             # Excludes secrets, build artifacts, state file
├── autocommit_config.json # (optional) User configuration overrides
├── requirements.txt       # Python dependencies
├── run_watcher.py         # CLI entry point
└── README.md              # This file
```

Stopping the Watcher

- **Interactive:** `Ctrl+C` — triggers clean shutdown.
- **Scheduled Task:** `Stop-ScheduledTask -TaskName "GitAutoCommit"` (PowerShell).
- The watcher drains any pending work before exiting.